

IP2LIVE RELEASE PLANNING

Deployment, Local Data, and Online Migration Guide

A repository-based assessment of the existing Windows export plus the hardened offline-storage, reporting, music, release, installer, browser, and online-database path.

Part I

What is ready now for offline Windows delivery

Part II

How to publish online and when a server database is needed

Evidence

Source code, exported Electron package, and existing runtime storage

Recommended first release

Fresh RPG Paper Maker export, hardened release preparation, then a signed Windows ZIP or installer

Prepared 21 August 2026 · Audit target: IP2Live project and IP2Live-1.1-Windows export

PURPOSE AND SCOPE

Executive answer

IP2Live can run fully offline and now has a hardened Windows data design: core saves, progress snapshots, monthly attempt journals, and archived reports are written under `%LOCALAPPDATA%\IP2Live`, while IndexedDB and localStorage remain useful mirrors. The existing `IP2Live-1.1-Windows` folder is an older audit baseline and must be freshly exported and prepared before distribution.

READY IN SOURCE

Offline Windows persistence. Saves are atomic and integrity-checked, a verified backup is retained, reports are archived with SHA-256 sidecars, and telemetry is journaled before being mirrored to IndexedDB.

OFFLINE BY DESIGN

Local attempts and reports. Passed, failed, cancelled, interrupted, and detailed mistake events are retained locally. The default report window is 90 days, raw evidence is included, and PDF/XLS/JSON files are archived to the managed Reports folder.

RE-EXPORT FIRST

The old Game.exe package is stale. It omitted all 11 custom music/effect assets. Create a fresh protected Windows export, run the supplied release-preparation script, then complete a clean-machine smoke test. A browser release still requires a separate web export.

The database question in one sentence

Putting the game online does not automatically require an online database. Keep IndexedDB if progress and reports may remain on one browser/device. Add a server database when you need accounts, cross-device continuation, centralized teacher reports, reliable backups, administrative dashboards, or authoritative assessment records.

Hardened implementation delivered with this guide

Area	Implemented behavior	Release consequence
Save/load	Atomic per-slot JSON, fsync, backup recovery, SHA-256 corruption detection, embedded custom progress metadata, and non-destructive migration from legacy <code>build\Saves</code> .	Install location may be read-only; the player data remains in <code>%LOCALAPPDATA%\IP2Live\Saves</code> .
Attempt history	Durable monthly JSONL journals are authoritative; IndexedDB is a query mirror. Events use profile/session/attempt/event IDs and pending writes are drained before export/quit.	Normal history is kept indefinitely and the reporting contract guarantees at least the requested three-month window.
Reports	PDF, SpreadsheetML <code>.xls</code> , and JSON evidence are atomically archived under <code>%LOCALAPPDATA%\IP2Live\Reports</code> with SHA-256 sidecars.	The UI reports real archive paths; export does not claim success if local archival fails.
Electron host	Stable <code>IP2Live</code> identity, sandbox, context isolation, no renderer Node.js, narrow validated IPC, blocked navigation/popups, and managed file-size/path limits.	Use the supplied preparation script after every fresh RPG Paper Maker export.
Music	Nine custom tracks and two effects are registered in RPG Paper Maker and played through its native <code>Data.Songs</code> /Howler path with retry and independent music/SFX volume.	A new protected export is mandatory so RPG Paper Maker embeds the registered audio bytes.

Important status boundary. All later references to the “current export” or `%APPDATA%\game` describe the audited `IP2Live-1.1-Windows` baseline. They are retained to explain the migration. The hardened behavior applies only after a fresh export is passed through `tools\prepare-windows-release.ps1`.

Recommended release sequence

- Fresh Windows export:** export again from RPG Paper Maker so the newly registered music and effects are embedded; do not distribute the stale `IP2Live-1.1-Windows` folder.
- Prepare and validate:** run `tools\prepare-windows-release.ps1` into a new destination. It installs the hardened host, synchronizes the plugin, excludes Git/developer saves, validates audio registrations, and writes a runtime hash manifest.
- Desktop production:** smoke-test save/restart/backup recovery, every music zone, 90-day reporting, standard-user install, update, and uninstall; then sign and publish the ZIP or installer.
- Browser pilot:** produce a separate RPG Paper Maker web export, clean it, test it from HTTPS, confirm report downloads and storage persistence, then publish as an itch.io HTML game or on a static web host.
- Online service:** retain IndexedDB as an offline cache/outbox and synchronize events through an authenticated HTTPS API into a server database.

Contents

1. Audit basis and confidence

2. Current package capability

3. Local data architecture

4. Attempts and export reports

5. Local capacity and durability

6. Offline delivery choices

7. itch.io downloadable workflow

8. Google Drive and direct delivery

9. Windows installer plan

10. Updates, backup, and migration

11. What “online” can mean

12. Browser and itch.io HTML deployment

13. IndexedDB in the browser

14. When to use an online database

15. Recommended online architecture

16. Data model and synchronization

17. Security and privacy

18. Phased implementation roadmap

19. Release acceptance checklists

Appendices and sources

PART I • OFFLINE AND WINDOWS DISTRIBUTION

1. Audit basis and confidence

This guide is based on three forms of evidence, not on assumptions about the screenshot alone:

- **Project source:** the inlined database manager, game manager, report manager, export screen, pause menu, and gameplay integrations in `Plugins/IP2Live_Core`.
- **Actual export:** `C:\Users\User\Desktop\IP2Live-1.1-Windows`, including its Electron `package.json`, `main.js`, preload, engine scripts, and build content.
- **Runtime evidence:** Chromium storage under `%APPDATA%\game` contains `IP2Live_Database`, a named profile, `session_start`, `attempt_start`, and `attempt_end` records.

Confidence boundary. Baseline local persistence is confirmed by source and real runtime data. The hardened storage, report, release, and music paths have automated tests, but a valid song-inclusive export was not yet available for a complete `Game.exe` clean-machine smoke test. That remains the final release gate.

480.1 MiB

Current full Windows
export

1,147

Files in the full export

123.3 MiB

Game build subtree

41.3 MiB

Unnecessary `.git`
data in build

The clean game build, excluding `.git`, is approximately 82.0 MiB across 609 files. The largest game payload is `songs.json` at about 66.5 MiB because audio is embedded as base64 data.

2. Baseline: what IP2Live-1.1-Windows can do

Why this baseline remains in the guide. It documents what users would receive if the old folder in the screenshot were distributed today and explains why migration and a fresh build are required. It is not the status of the newly hardened source.

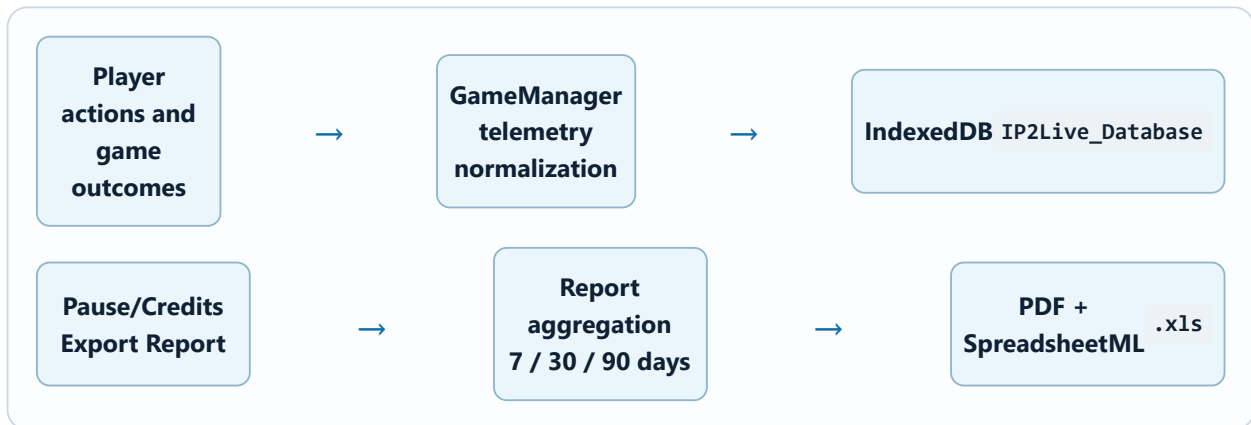
Capability	Current state	Operational meaning
Run without the RPG Paper Maker editor	YES	The exported Electron runtime is self-contained. Users do not need Node.js, Electron, or RPG Paper Maker installed.
Run offline	YES	Core game, local data, and report generation have no required server dependency.
Send only <code>Game.exe</code>	NO	The executable depends on <code>resources</code> , <code>locales</code> , DLLs, PAK files, and other neighboring runtime files.
Keep named profiles	YES	Profiles are keyed by <code>infiltratorName</code> in IndexedDB.
Keep detailed attempt telemetry	YES	Attempt starts, ends, mistakes, outcomes, timings, stage data, retries, and gameplay-specific evidence are stored.
Save/load game slots	YES, PATH RISK	Core save JSON is written under the exported app's <code>build\Saves</code> ; progress metadata is also mirrored in <code>localStorage</code> and IndexedDB.
Export PDF and spreadsheet report	IMPLEMENTED	The menu offers PDF, Excel, or both. "Excel" is SpreadsheetML XML saved with an <code>.xls</code> extension, not native <code>.xlsx</code> .
Safe conventional installer	NOT YET	The current save path can become unwritable or be removed during upgrades/uninstall.
Play directly in a website	SEPARATE BUILD	A dedicated web export is required. The desktop shell relies on Electron APIs.
Central teacher dashboard	NO	Data stays on each Windows account/browser unless the player manually exports and sends a report.

How to distribute the package correctly

1. Make a release copy of the exported folder. Do not clean or alter the only master copy.
2. Remove release-inappropriate content from the copy—at minimum `resources\app\build\.git`, `.gitignore`, and internal verification documents—then smoke-test.
3. Keep every Electron runtime file and the entire `resources` and `locales` directories.
4. Create one versioned ZIP, for example `IP2Live-1.1-Windows-x64.zip`.
5. Tell users to extract the ZIP completely, then run `Game.exe`. Do not run it from inside the ZIP viewer.
6. Publish a SHA-256 checksum and a short README with controls, minimum Windows version, report instructions, and support contact.

Do not ship the current export unchanged. It contains the project's Git history inside `resources\app\build\.git`. That adds 459 files and about 41.3 MiB and may expose deleted or historical source content. Removing it also brings the game-content file count from 1,068 to 609.

3. Local data architecture: baseline and hardened release



Audited 1.1 baseline locations

Data	Technology and location	Current purpose	Main risk
Profiles	IndexedDB store <code>profiles</code> <code>%APPDATA%\game\IndexedDB</code>	Name, creation/update dates, last save slot, map, play time, and per-slot progress snapshots.	Name is the primary key; two people using the same Windows account and same name merge into one logical profile.
Telemetry	IndexedDB store <code>telemetry</code> indexed by name, stage, timestamp, gameplay, event type	Append-style attempt and mistake history used to build reports.	Local, user-clearable, unencrypted, not centrally backed up, and not authoritative.
Sessions store	IndexedDB store <code>sessions</code>	Schema exists.	No code was found writing session rows to this store; reports infer sessions from telemetry <code>sessionId</code> .
Slot snapshot cache	<code>localStorage</code> key <code>IP2Live_SlotProgress_v1</code> <code>%APPDATA%\game\Local Storage</code>	Small map/quest/position metadata used to restore custom progress.	Web Storage is much smaller than IndexedDB and can be cleared with site/app data.
Core RPG save	JSON at <code>resources\app\build\Saves\{slot}.json</code>	Engine-level party, variables, inventory, map, hero position, and other game state.	Lives in the program tree rather than the user-data directory.
Exported reports	Browser/Electron download destination, normally Downloads or the configured browser download path	Portable copy for learner/teacher sharing.	Contains identifiable performance data and must be handled accordingly.

Hardened Windows release locations

Data	Authoritative location	Durability and role
Core save plus progress metadata	%LOCALAPPDATA%\IP2Live\Saves\{slot}.json	Atomic replacement, fsync, verified .bak recovery, bounded JSON, and SHA-256 corruption detection. Saves do not expire.
Progress sidecar	%LOCALAPPDATA%\IP2Live\Data\progress-slot-{slot}.json	Recoverable custom snapshot; the same snapshot is embedded in the core save transaction so mirrors cannot silently outrank durable state.
Attempt evidence	%LOCALAPPDATA%\IP2Live\Telemetry\YYYY-MM.json1	Append-only monthly journal with record hashes and flush-before-export. Kept indefinitely by default, guaranteeing at least 90 days.
Profiles/query cache	IndexedDB under %LOCALAPPDATA%\IP2Live\Browser	Profile, session, telemetry, and indexes for fast UI/report queries; rebuildable from durable evidence where applicable.
Reports	%LOCALAPPDATA%\IP2Live\Reports	PDF, .xls , and JSON evidence plus .sha256 sidecars; downloaded copies are optional conveniences.

Why there are three persistence layers

The app combines the RPG Paper Maker core save file with custom restore metadata and educational telemetry. IndexedDB is the correct layer for structured attempt history; localStorage is suitable only for the small snapshot map; the core save JSON is an engine requirement. The architecture is workable, but all three layers must be migrated and tested as a unit when packaging changes.

4. Does it record all attempts and export them?

Hardened answer: normal gameplay paths record starts, mistakes, and one terminal outcome; interrupted starts are preserved as unassessed evidence instead of disappearing. A forced power loss can still interrupt the final physical write in any local application, which is why the design uses an append-only journal, fsync, recovery, event IDs, and an audit sheet.

Question	Finding
Are gameplay launches recorded?	Yes. <code>attempt_start</code> is written when a gameplay node opens.
Are successes and failures recorded?	Yes. Both close as <code>attempt_end</code> with passed/failed outcome and gameplay-specific metrics.
Are cancellations recorded?	Yes. Cancellation closes an <code>attempt_end</code> marked cancelled and unassessed. Six previously wrong or missing gameplay cancellation paths were corrected.
Are mistakes/retries recorded?	Yes where gameplay calls the central mistake handler. Raw <code>attempt_mistake</code> events and summarized retry/mistake data are supported.
Does the report show incomplete/crashed attempts?	Yes. An orphaned <code>attempt_start</code> is reconciled as interrupted and remains unassessed; it is also visible in the raw Event Audit and JSON evidence.
What history is exported?	The menu retains 7/30/90-day choices and defaults to 90 days. The durable journal is retained indefinitely by default, while the normal reporting guarantee is at least three months.
Is the latest event drained before export/quit?	Yes during controlled export and quit paths. Pending journal and IndexedDB writes are awaited and storage failures fail the export instead of reporting false success.

What the report already provides

- Player name, selected date window, generated date, sessions, active days, and play duration.
- Gameplay instances, assessed attempts, passes, failures, cancellations, repetitions, rounds, retries, and mistakes.
- Accuracy, completion rate, average/median/best completion time, trends, mastery indicators, competency summaries, and suggested intervention.
- Attempt history with stage, level, map, quest, objective, gameplay, timing, outcome, recovery/rollback, security alerts, and evidence payload.
- Daily, stage, quest, gameplay, competency, and step/mistake breakdowns.

Meaning of “all attempts.” The hardened event audit includes complete and interrupted attempts inside the selected window and archives raw JSON evidence. The menu intentionally promises a 90-day default rather than an all-time report. Local hashes detect accidental corruption; they do not make a learner-controlled device tamper-proof.

5. How capable is the app’s local storage?

For ordinary educational telemetry, capacity is not the limiting factor. IndexedDB is designed for structured, indexed datasets much larger than localStorage. The current records are JSON-like objects and should allow many thousands of attempts on a normal device. The bigger concerns are durability, identity, backup, and trust.

Dimension	Assessment	Recommendation
Offline operation	Strong	Keep IndexedDB even if a server is later added; it supports offline-first play.
Volume	Strong for telemetry	Monitor <code>navigator.storage.estimate()</code> ; add retention/export only if actual usage grows unusually.
Persistence	Strong on hardened Windows; best-effort on web	Windows uses managed files with backup recovery. Request persistent browser storage for a future web build.
Cross-device use	None	Add authentication and server synchronization if required.
Multi-user separation	Improved	New profiles receive immutable UUIDs; display names remain user-facing labels. Use authenticated server IDs for cross-device accounts.
Integrity	Weak	Do not treat a locally generated report as tamper-proof. Use server-received events for formal assessment.
Backup	Local backup plus evidence archive	Each save keeps a verified backup and each report has a checksum. Add full bundle import/export or online sync for off-device disaster recovery.
Privacy	Local by default	This minimizes transmission, but device users can inspect the data. Define retention and deletion controls.

Browser storage is origin-scoped and best-effort by default. Users can erase it, private browsing normally removes it at session end, and browsers may evict non-persistent data under storage pressure. Web Storage is commonly limited to about 10 MiB per origin, while IndexedDB receives a much larger browser-managed quota. These are reasons to keep high-volume telemetry in IndexedDB and small metadata only in localStorage.

6. Offline delivery options

Channel	Best for	Strengths	Weaknesses	Recommendation
itch.io downloadable ZIP	Public or controlled pilots	Game page, platform labeling, versioned uploads, itch app support, Butler patching.	Unsigned executables may warn; users still download/extract unless using itch app.	Best immediate option.
Google Drive / OneDrive / Dropbox	Small private cohorts	Fast to share, familiar permissions.	No game-specific updates, weak version control, link/access issues, no platform metadata.	Acceptable pilot backup.
USB / LAN / school LMS	Offline lab deployment	No internet dependency, controlled audience.	Manual updates and data collection.	Good for supervised labs.
Portable EXE folder	Technical users	No installation; current format already works.	Large folder and easy to separate files accidentally.	Always deliver as one ZIP.
Installer (Inno/NSIS/MSIX)	Broad Windows release	Shortcuts, uninstall, clean presentation, version metadata.	Storage and identity are now prepared; installer configuration, signing, upgrade/uninstall policy, and clean-machine QA remain.	After the hardened ZIP passes.
itch.io HTML/browser	Zero-install access	One click, cross-device browser access.	Separate build; browser storage and performance constraints.	Separate web pilot.

7. Publishing the hardened game as an itch.io downloadable

Release preparation

1. Export a fresh Windows build from the intended tagged source revision.
2. Run `tools\prepare-windows-release.ps1` with the fresh export as `-SourceExportPath`, a new empty destination, `-ProjectRoot`, and the release version. Do not manually patch the export in place.
3. Verify `Game.exe` starts on a clean Windows user account with no RPG Paper Maker installation.
4. Create a profile, complete a pass, a failure, and a cancellation, save to a non-default slot, restart, load, and export both formats.
5. Close the game, ZIP the whole folder, extract that ZIP to a new location, and repeat the launch/load test.
6. Scan the release with current endpoint protection, calculate SHA-256, and archive the exact released ZIP and source tag.

itch.io page setup

1. Create a project, choose **Downloadable**, and upload the ZIP directly or with Butler.
2. Mark only Windows for this file, since that is the platform actually exported and tested.
3. Provide version, approximate download/extracted size, 64-bit status if applicable, controls, save/report behavior, and known limitations.
4. For private testing, keep the page restricted or use itch.io download keys. For a public release, add 3–5 screenshots and accurate metadata.
5. For updates, keep the same project and Windows channel so players receive the new build through the itch app.

Butler workflow

```
butler login
butler push "C:\Release\IP2Live-1.1-Windows" YOUR_ACCOUNT/ip2live:windows
```

Butler accepts a directory or ZIP, creates compressed patch data, and efficiently uploads only changed content on later pushes. Its current uncompressed build limit is far above this game's size. Do not place API credentials in source control or build logs.

Licensing check. RPG Paper Maker states that free use is for non-commercial releases and that commercialized games require its commercial license. Confirm the engine license and every font, audio, image, video, and third-party asset license before charging money, accepting revenue tied to access, or distributing commercially.

8. Google Drive and other direct-delivery methods

Direct links are technically sufficient because the build is self-contained. They are less convenient for updates, so use a disciplined release layout:

- One immutable ZIP per version; never silently replace a file with different contents under the same filename.
- A separate `README.txt` or landing page with "download → extract all → run Game.exe".
- Published SHA-256 checksum, file size, release date, change log, and support contact.
- Read-only sharing permissions and a tested anonymous/student-access link.

- A retention policy for older packages and a clear label for the currently supported version.

Update behavior. Hardened releases share `%LOCALAPPDATA%\IP2Live`, so installing or extracting a newer version elsewhere does not strand new-format saves or reports. The first hardened launch can import normal JSON saves found in its legacy `build\Saves`; moving from a different old portable folder still requires an explicit migration/backup step.

9. Windows installer plan

Installer readiness status

1. **Implemented — identity:** the prepared package uses the stable product/package identity `IP2Live` and an isolated Chromium directory.
2. **Implemented — per-user data:** saves, progress, telemetry, reports, and browser state live under `%LOCALAPPDATA%\IP2Live`, outside program files.
3. **Implemented — legacy core saves:** first launch imports valid normal JSON slots found in the legacy build folder without deleting or overwriting the source. Back up and explicitly migrate the old generic `%APPDATA%\game` browser data if preserving a 1.1 profile history is required.
4. **Implemented — release hygiene:** the preparation tool excludes embedded `.git` directories and developer save JSON, exactly synchronizes the current plugin, validates songs, and writes runtime hashes.
5. **Implemented — host hardening:** renderer Node.js is disabled, context isolation and sandboxing are enabled, IPC is narrow and sender-validated, paths are never accepted from renderer storage calls, and unexpected navigation/popups are blocked.
6. **Still required — product release:** configure the installer, test update/downgrade/uninstall policy, sign and timestamp the executable/installer, and verify on clean standard-user devices.

Pragmatic installer choice

Because RPG Paper Maker already produced a complete Electron directory, Inno Setup is a straightforward wrapper. NSIS is equally viable; MSIX offers stronger Windows integration but has stricter packaging rules. A per-user installer avoids an elevation prompt, but it does not remove the need to keep user data outside the application directory.

```
; Illustrative Inno Setup skeleton – build from a CLEAN release folder
[Setup]
AppId={{B57A9A5C-9E9F-4AC7-A11A-CHANGE-ONCE-AND-KEEP}}
AppName=IP2Live
AppVersion=1.1.0
DefaultDirName={localappdata}\Programs\IP2Live
DefaultGroupName=IP2Live
PrivilegesRequired=lowest
OutputBaseFilename=IP2Live-1.1.0-Setup
Compression=lzma2
SolidCompression=yes
WizardStyle=modern

[Files]
Source: "release\IP2Live-1.1-Windows\*"; DestDir: "{app}"; \
Flags: ignoreversion recursesubdirs createallsubdirs

[Icons]
Name: "{group}\IP2Live"; Filename: "{app}\Game.exe"
Name: "{autodesktop}\IP2Live"; Filename: "{app}\Game.exe"; Tasks: desktopicon

[Tasks]
Name: desktopicon; Description: "Create a desktop shortcut"
```

Replace the example GUID once, then preserve it for every upgrade. Set publisher, URL, icon, license, architecture, and version-resource fields for production. Compile from a clean directory and sign the resulting installer.

Installer behavior to specify

- Upgrades replace program files but never delete profiles, saves, or telemetry.
- Uninstall removes application files; user data remains by default, with an explicit optional “remove my data” action.
- A first-launch migration log records what was found and copied without containing sensitive attempt details.
- The About screen exposes version, data location, report export, backup, privacy, and support information.

10. Updates, backups, and migration

Hardened-release manual backup

1. Exit IP2Live completely so its journals and browser database are closed.
2. Back up the whole `%LOCALAPPDATA%\IP2Live` directory. This contains saves, backups, progress, telemetry, report archives, and browser mirrors.
3. Verify the copied save/report SHA-256 sidecars where applicable and protect the backup because reports contain identifiable performance information.
4. For a one-time 1.1 migration, also preserve the old export's `resources\app\build\Saves` and `%APPDATA%\game` before changing identity.

Preferred future backup format

Add an in-game “Export My Data” command that creates one versioned bundle containing profile UUID, slot snapshots, engine save, telemetry event rows, schema version, export date, and a checksum. Add a matching import flow that validates the schema and shows a conflict preview.

Old profile-history migration is a separate operation. The prepared host intentionally stops using the collision-prone `%APPDATA%\game` identity. Preserve that directory before upgrading existing pilot users; the release tool migrates core JSON saves, not an arbitrary generic Chromium profile.

PART II • BROWSER DEPLOYMENT AND ONLINE DATA

11. “Deploy online” can mean three different products

Model	Game delivery	Data	Server database?
A. Download page	Players download the Windows ZIP from itch.io or another website.	Same local AppData/save behavior as today.	No. The website is only a delivery channel.
B. Browser game, local-only	Game runs as HTML/JavaScript in the browser.	IndexedDB/localStorage remain on that browser and origin.	No. Useful for casual or private single-device play.
C. Connected learning service	Desktop and/or browser clients authenticate and sync.	Local offline cache plus centralized server records.	Yes. Needed for accounts, dashboards, cross-device data, and reliable collection.

12. Browser and itch.io HTML deployment

Do not upload the Windows export as an HTML game. Its `index.html` is the renderer of an Electron app; the package uses a preload and `window.ipcRenderer` for desktop file I/O. Use RPG Paper Maker’s separate “Deploy for web browser” output.

Build and test workflow

1. In RPG Paper Maker, export **Deploy for web browser** to a new empty directory.
2. Confirm `index.html` is at the ZIP root, not inside an extra parent folder.
3. Remove `.git` and internal files. Measure extracted size, individual file sizes, file count, and longest path.
4. Serve locally through HTTP—not by double-clicking `index.html`—and test Chrome and Edge first, then Firefox. Test at 1280×720 and smaller viewports.
5. Check the developer console and network panel for missing resources, case mismatches, absolute paths, mixed HTTP content, autoplay restrictions, codec failures, and cross-origin errors.
6. Verify every gameplay module, save/load, keyboard/mouse behavior, report export, reload persistence, browser restart persistence, private-window behavior, and storage clearing.
7. Zip only the web output contents and upload it as an itch.io **HTML Game**.

Current itch.io HTML constraints to design for

Constraint	itch.io requirement	IP2Live implication
Entry point	<code>index.html</code> inside a ZIP	Use the web export root.
Extracted files	Maximum 1,000	The current desktop build subtree has 1,068 because it contains <code>.git</code> ; without <code>.git</code> it has 609. Measure the actual web export.
Extracted total	Maximum 500 MiB by default	The cleaned game payload is about 82 MiB, but the web build must be measured independently.
Single extracted file	Maximum 200 MiB	The current largest game file is 66.5 MiB; within the limit, but expensive to parse/load.
Path length	Maximum 240 characters	Check deeply nested gameplay paths.
Filenames	Case-sensitive, UTF-8	Windows success does not prove web-host success.
Resource URLs	Relative paths; external APIs must use HTTPS	Audit plugin fetches and future API base URL.

Performance priorities for the web build

- The 66.5 MiB base64 audio JSON expands in memory and must be parsed. Prefer web-appropriate streamed/compressed audio assets if RPG Paper Maker supports that export mode.
- Check the `.mov` title video codec in Chrome, Edge, Firefox, and Safari. Container extension alone does not guarantee browser playback.
- Lazy-load large audio/video and gameplay assets where practical; show a clear loading indicator and require a click before audio starts.
- Do not enable mobile support until touch controls, layout, memory use, and real devices are tested.

13. What happens to IndexedDB when the game is online?

The same IndexedDB code can operate in a browser because it uses standard Web APIs. Its meaning changes:

- Data belongs to the site's **origin**—scheme, hostname, and port—not to an IP2Live account.
- A different browser profile, computer, hostname, protocol, or port receives a different empty database.
- Clearing site data, private browsing, storage pressure, browser policies, or corruption can delete it.
- An itch.io game runs inside an iframe and browser privacy policies may affect third-party storage. Test the exact published page, not only localhost.
- IndexedDB does not synchronize itself with a server database. Sync logic must be designed and implemented.

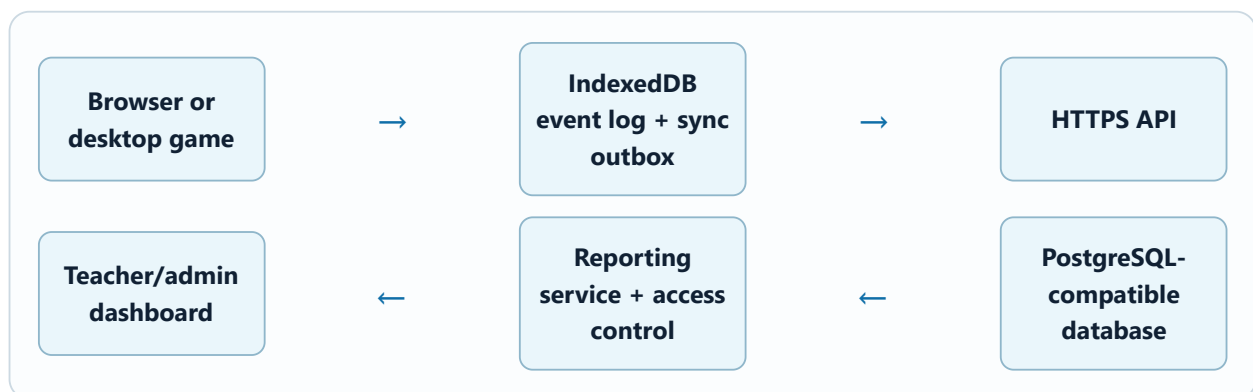
Local-only web hardening

1. Call `navigator.storage.persist()` after explaining why persistence is useful.
2. Show storage status and approximate usage from `navigator.storage.estimate()`.
3. Add full data export/import and an "erase my local data" control.
4. Show a clear warning in private browsing or when persistence is unavailable.
5. Use a stable production domain/origin and test data continuity across deployments.

14. When an online database becomes necessary

Requirement	Local IndexedDB only	Server database
Play offline on one device	Excellent	Optional
Continue on another device	Cannot	Required
Teacher sees reports without student sending files	Cannot	Required
Recover after browser/device loss	Only manual backup	Required for automatic recovery
Multiple students on shared computers	Name-based separation is weak	Recommended with authenticated IDs
Formal/authoritative attempt record	User can alter/clear local data	Required, with server-side validation/audit
Aggregate research/analytics	Manual collection	Required, subject to consent and governance
Simple public game with private local progress	Sufficient	Unnecessary complexity

15. Recommended online architecture



Keep the client offline-first. Each event is saved locally first. A background sync worker sends pending events when connectivity is available. The API authenticates the player, validates the payload, assigns/accepts an idempotency key, stores the event once, and acknowledges it. The client then marks the outbox item synchronized.

Do not connect the game directly to the database

Database credentials embedded in JavaScript or an Electron renderer are recoverable by users. Place authorization, validation, rate limiting, data ownership checks, and schema evolution behind an HTTPS API or a managed backend that enforces equivalent server-side policies.

Deployment components

- **Static game host:** itch.io HTML hosting or another HTTPS static/CDN host.
- **API:** versioned endpoints such as `/v1/events:batch`, `/v1/progress`, and `/v1/reports`.
- **Database:** relational storage is a strong fit because users, sessions, attempts, mistakes, stages, and reports have clear relationships.

- **Authentication:** school SSO, managed accounts, or invitation codes mapped to immutable user IDs. Never use display name as identity.
- **Dashboard:** separate teacher/admin web application with role-based access and server-generated reports.
- **Observability:** service health and sync failure metrics without logging sensitive payloads unnecessarily.

16. Suggested server data model and sync contract

Entity	Key fields	Notes
users	user_id UUID, external identity, display name, status	Display name may change; UUID does not.
devices	device_id, user, first/last seen, client version	Supports diagnostics and revocation without fingerprinting more than needed.
sessions	session_id, user, device, started/ended, client version	Populate deliberately rather than relying only on inferred telemetry.
attempts	attempt_id, event ID, user/session, gameplay, quest, outcome, timing, metrics	Append-first; unique event ID prevents duplicate sync.
mistake_events	event ID, attempt ID, step, issue, submitted/expected, timestamp	Keep detailed evidence separate from the attempt summary.
progress_snapshots	user, slot, revision, map, quest state, save payload, updated time	Versioned mutable state; define conflict policy.
report_exports	requester, subject, scope, generated time, format, audit reference	Useful if reports are official records.
consents / deletions	subject, policy version, action, timestamps	Supports governance where required.

Event envelope

```
{
  "eventId": "uuid-generated-on-client",
  "schemaVersion": "telemetry-v3",
  "userId": "server-issued-uuid",
  "deviceId": "client-installation-uuid",
  "sessionId": "uuid",
  "eventType": "attempt_end",
  "occurredAt": "2026-08-21T13:09:58.000Z",
  "clientVersion": "1.2.0",
  "payload": { "attemptId": "uuid", "gameplayId": "...", "outcome": "passed" }
}
```

Synchronization rules

- **Idempotency:** server enforces uniqueness on `eventId`; retrying the same batch never creates duplicates.
- **Ordering:** retain client timestamp and sequence, but also store server receipt time. Do not trust client clock for authorization.
- **Batching:** send small batches, compress where supported, and back off after failures.
- **Append vs mutable:** attempts/mistakes are append-only; progress snapshots are versioned and need conflict resolution.
- **Acknowledgement:** remove nothing from the local outbox until the server confirms accepted event IDs.
- **Schema evolution:** accept at least the current and previous event version during rolling updates.

- **Deletion:** a server deletion request must also tell all signed-in clients to clear the corresponding local data.

17. Security, privacy, and trust boundaries

Desktop package

- Keep external content out of the privileged Electron renderer. The current `nodeIntegration: true / contextIsolation: false` combination is acceptable only as a legacy local-content constraint, not as a pattern for loading remote pages.
- Validate every IPC request, especially file paths. The current save IPC resolves a renderer-provided relative path against the app directory.
- Remove DevTools shortcuts in production or gate them behind a deliberate support mode.
- Use a current supported Electron line, track vulnerabilities, sign releases, and publish checksums.

Online service

- HTTPS only; never ship database administrator keys in the client.
- Use role-based authorization: student sees own records; teacher sees assigned groups; administrator scope is explicit and audited.
- Validate sizes, types, allowed gameplay IDs, timestamps, and ownership on the server. Rate-limit event submission and report generation.
- Encrypt data in transit and at rest, manage secrets outside source control, back up the database, and test restoration.
- Collect the minimum necessary personal information. Define purpose, consent/notice, retention, correction, export, and deletion procedures before a school rollout.
- Separate research analytics from operational records and de-identify exports where identity is unnecessary.

Trust statement. A PDF produced entirely from local IndexedDB proves what the local software reported, not that the data was never modified. For graded or compliance-sensitive use, generate the official report from authenticated server-received events and include report/audit identifiers.

18. Phased implementation roadmap

Phase	Deliverable	Exit criteria
0. Release hygiene	Clean Windows export; release README; checksum; source tag.	No <code>.git</code> /internal files; clean-user smoke test passes.
1. Offline pilot	itch.io downloadable Windows ZIP; optional restricted Drive mirror.	Pass/fail/cancel, save/load, restart, and PDF/XLS export pass on at least two Windows devices.
2. Desktop production	Stable identity, user-data saves, migration, installer, signing.	Install/update/uninstall matrices prove no unintended data loss; signatures verify.
3. Local-only web pilot	Dedicated RPG Paper Maker web build on HTTPS/itch.io.	Size limits, asset loading, gameplay, persistence, report download, and privacy-mode behavior pass.
4. Sync foundation	UUID identity, local outbox, authenticated batch API, server attempts.	Offline play and reconnect sync are idempotent; duplicate/loss tests pass.
5. Connected learning service	Progress sync, teacher dashboard, server reports, governance controls.	Role/access, backup restore, deletion/export, monitoring, and security review pass.

19. Release acceptance checklists

Windows portable

- ☐ Fresh export from tagged source
- ☐ No `.git` or private/internal files
- ☐ Entire folder zipped
- ☐ Launch from extracted ZIP copy
- ☐ Clean Windows user test
- ☐ Offline launch test
- ☐ All gameplay modules smoke-tested
- ☐ Pass/fail/cancel telemetry verified
- ☐ Save slots survive restart
- ☐ PDF and XLS opened and checked
- ☐ Checksum and malware scan complete
- ☐ License inventory approved

Browser/itch HTML

- ☐ Separate web export used
- ☐ `index.html` at ZIP root
- ☐ $\leq 1,000$ files, ≤ 500 MiB extracted
- ☐ No individual file > 200 MiB
- ☐ Relative, case-correct paths
- ☐ HTTPS-only external calls
- ☐ Chrome/Edge/Firefox tested
- ☐ Exact itch iframe tested
- ☐ Reload/restart persistence tested
- ☐ Storage clear/private mode explained
- ☐ Report downloads verified
- ☐ Audio/video compatibility verified

Installer/update

- ☐ Stable app/package/installer IDs
- ☐ User data outside app directory
- ☐ Old portable data migration tested
- ☐ Upgrade over previous version tested
- ☐ Downgrade behavior documented
- ☐ Uninstall preserves data by default
- ☐ Standard-user install tested
- ☐ Paths with spaces/non-ASCII tested
- ☐ Executable and installer signed
- ☐ SmartScreen/SAC behavior checked
- ☐ Recovery/backup guide published
- ☐ Support diagnostics available

Online database

- ☐ Immutable user/device/event IDs
- ☐ No secrets in client
- ☐ Authorization tested by role
- ☐ Server-side validation and limits
- ☐ Idempotent retry/deduplication
- ☐ Offline/reconnect test
- ☐ Clock/order conflict test
- ☐ Backup restore rehearsal
- ☐ Retention/deletion/export flows
- ☐ Consent/privacy notice approved
- ☐ Monitoring and incident runbook
- ☐ Server-generated reports reconciled

APPENDICES

Appendix A. Implementation status and remaining backlog

Priority	Backlog item	Reason
Done	Exclude <code>.git</code> and developer saves from prepared releases.	Automated by the release tool and covered by an integration test.
Done	Move core saves out of <code>resources\app\build\Saves</code> .	Managed per-user save files, backups, and migration are implemented.
Done	Set stable IP2Live identity and isolate browser data.	The prepared host uses <code>%LOCALAPPDATA%\IP2Live</code> .
Done	Reconcile interrupted attempts and add raw report evidence.	Event Audit plus JSON evidence covers incomplete starts; 90 days remains the promised default window.
Done	Await telemetry/report flush and expose failures.	Controlled export and quit paths drain pending storage writes.
Done	Add immutable UUID profile IDs for new profiles.	Display-name collisions no longer define event ownership.
P1	Add full data export/import and erase controls.	Backup, migration, and privacy.
Done	Harden Electron preload, IPC, and navigation.	Sandboxed CommonJS preload, context isolation, trusted senders, fixed destinations, and blocked popups/navigation are implemented.
P2	Use native <code>.xlsx</code> or label the current format precisely.	Current XML-in- <code>.xls</code> may produce format warnings.
P2	Optimize base64 audio and validate video codec for web.	Startup memory and browser compatibility.

Appendix B. Hardened release-preparation command

Run this from the project root after RPG Paper Maker produces a fresh protected Windows export. The destination must not already exist; the source export is left unchanged.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
-File .\tools\prepare-windows-release.ps1 `
-SourceExportPath 'C:\Exports\IP2Live-Windows' `
-DestinationPath 'C:\Release\IP2Live-Prepared' `
-ProjectRoot . `
-ReleaseVersion '1.1.0'
```

The command fails before creating a destination if any registered custom track is absent from the exported `build\songs.json`. Do not bypass that guard; re-export so RPG Paper Maker embeds the audio.

Appendix C. Sources

Project and build evidence

- `Plugins/IP2Live_Core/code.js` : database name/version, stores, indexes, and IndexedDB CRUD helpers.
- `Plugins/IP2Live_Core/modules/game_manager.js` : telemetry lifecycle, gameplay metrics, localStorage snapshots, profile persistence, save/load bridge, and report export.
- `Plugins/IP2Live_Core/modules/report_manager.js` : local-only aggregation, attempt filtering, PDF construction, SpreadsheetML export, and download behavior.
- `Plugins/IP2Live_Core/modules/screens/export-report.js` : 7/30/90-day scopes and PDF/Excel/Both choices.
- `Plugins/IP2Live_Core/modules/desktop_storage.js` : engine save hooks, progress snapshots, telemetry journals, report archival, and browser fallback.
- `deployment/windows` and `tools/prepare-windows-release.ps1` : hardened Electron host, bounded storage service, secure preload bridge, and repeatable release construction.
- Exported `resources/app/package.json` and `main.js` : Electron 43.2.0, generic package name `game`, desktop window and IPC settings.
- Exported RPG Paper Maker engine `Core/Game.js`, `Common/Paths.js`, `Common/IO.js` : core save path and desktop save IPC.
- Runtime Chromium profile `%APPDATA%\game` : actual IP2Live database/profile/session/attempt evidence.

Current external documentation (accessed 21 August 2026)

- [itch.io — Uploading HTML5 games](#): ZIP structure, iframe behavior, file/size/path limits, relative paths, HTTPS, and case sensitivity.
- [itch.io — Pushing builds with Butler](#): push syntax, channels, patches, and upload limits.
- [itch.io — Your first itch.io page](#): downloadable projects and platform uploads.
- [itch.io — Content creator quality guidelines](#): accurate platform classification and direct uploads.
- [RPG Paper Maker — Official documentation repository](#): desktop and web-browser deployment modes.
- [RPG Paper Maker — Official site](#): supported deployment targets and commercial-license statement.
- [Electron — Application packaging](#): distributing the complete runtime folder and packaging tools.
- [Electron — Code signing](#): signing distributed Electron applications.
- [Electron — Security checklist](#): context isolation, Node integration, sandboxing, CSP, IPC, and navigation.
- [Microsoft — Code signing for Smart App Control](#): trusted RSA signing and SignTool guidance.
- [MDN — Storage quotas and eviction criteria](#): best-effort/persistent storage, quotas, private mode, clearing, and eviction.
- [MDN — IndexedDB key characteristics](#): offline capability, origin scope, clearing risks, and lack of automatic server synchronization.
- [Inno Setup — Non-administrative install mode](#) and [AppId](#): per-user installation considerations and stable upgrade identity.

Final recommendation

Release IP2Live first as a hardened Windows downloadable ZIP on itch.io. Create a fresh protected export so all custom audio is embedded, prepare it into a new destination with the supplied script, perform the clean-machine save/restart/report/music matrix, then sign and publish it. Google Drive is suitable for a controlled pilot mirror. The old screenshot build must not be distributed as the finished release.

For browser availability, make a separate RPG Paper Maker web export and initially keep IndexedDB local. Add a server database only when the product requirements include accounts, cross-device progress, central teacher visibility, reliable backup, or authoritative assessment. When that point arrives, do not discard IndexedDB: turn it into the offline event log and synchronization outbox behind an authenticated HTTPS API.

IP2Live Deployment, Local Data, and Online Migration Guide · Repository/build audit dated 21 August 2026. Validate external platform limits and legal/licensing requirements again at each release.